

Long-context prefill: a literature review of sparse attention and KV cache compression (2023–2026)

The dominant story of long-context LLM efficiency in 2024–2026 is that **post-hoc inference tricks have been overtaken by architectural choices baked into pretraining**. Methods that began as inference-only patches — sparse attention patterns, KV-cache eviction, low-rank projections — have either matured into production kernels (MInference, FP8/INT8 KV cache, PagedAttention) or been absorbed into new architectures (DeepSeek's MLA and NSA, Moonshot's MoBA). The result is a two-layer stack: an architectural layer that pre-compresses (MLA) and pre-sparsifies (NSA/MoBA/DSA), and a post-hoc layer (MInference, RetrievalAttention, RazorAttention, KV quantization) that complements rather than competes. This review traces both layers, the benchmarks that justify them, and where they sit in real serving systems.

The stakes are concrete. At 1 M tokens with LLaMA-3-8B, prefill alone takes roughly 30 minutes on a single A100; attention is **$O(n^2)$ and compute-bound**, while decode is memory-bandwidth-bound on the loaded KV cache. These two regimes need fundamentally different optimizations, which is why the field has bifurcated into prefill-focused sparse attention and decode-focused KV compression — **and why the most interesting recent work (NSA, MoBA, FastKV, YOCO) tries to optimize both simultaneously.** (arxiv +2)

The prefill bottleneck and why it differs from decode

Long-context inference has two phases with opposite resource profiles. **Prefill** processes the full prompt in parallel: QK^T and $\text{softmax}(\cdot)V$ are quadratic in sequence length n in both FLOPs and memory traffic, and on modern GPUs the operation is compute-bound. This phase determines **time-to-first-token (TTFT)**, which dominates user-perceived latency in RAG, repository-level coding, and agentic workloads. **Decode** is autoregressive: each step is a query-of-1 matmul against the entire KV cache, making it memory-bandwidth-bound. Optimizations therefore split: skipping block matmuls reduces prefill latency directly, while skipping cache loads reduces decode HBM traffic. **MInference targets only prefill; H2O and SnapKV originated in decode**; this is the central reason the literature is fragmented and why **2025 architectural methods (NSA, MoBA) emphasize that they are sparse in both phases.**

(arXiv + 2)

A second constraint shapes every method: **sparsity must be block-structured** (typically 64×64 or 128×128 tiles) to map onto Hopper's WGMMMA / Blackwell tensor cores. **Random token-level sparsity wastes the entire MMA tile and yields essentially no speedup on modern GPUs. This single hardware fact explains why BigBird, SeerAttention, NSA, MoBA, and XAttention all operate at block granularity, and why FlashAttention-2/3 (Dao 2023; Shah et al., NeurIPS 2024) is the universal substrate underneath.**

Sparse attention for prefill: a taxonomy

Sparse attention research falls along five axes that together define a method's deployment

sparse attention research runs along two axes that together define a method's deployment profile.

Selection mechanism divides query-agnostic patterns (sliding window, dilated, A-shape, Tri-Shape) from query-aware methods that recompute the sparse set per prompt (MInference, FlexPrefill, XAttention, SeerAttention, NSA, MoBA, Quest). **Training requirement** separates training-free / plug-and-play methods (MInference, FlexPrefill, XAttention, SpargeAttn, HiP, Quest, RetrievalAttention, Twilight) from gate-finetuned methods (SeerAttention) and natively pretrained methods (NSA, MoBA, InfLLM-V2, LongNet). **Granularity** ranges from layer-level alternation (Gemma-2's mixed SWA/full) to **head-level pattern assignment** (MInference) to block-level (most modern work) and token-level (DeepSeek's V3.2 DSA, Twilight's nucleus pruner). **Pattern source** can be predefined templates (StreamingLLM, BigBird, Star Attention) or content/affinity-based (SeerAttention's gate, MoBA's router, XAttention's antidiagonal sum, FlexPrefill's JSD). **Calibration timing** is offline-per-head (MInference, MMInference, Star Attention's anchor design) or fully online-per-prompt (FlexPrefill, XAttention, SpargeAttn, NSA, MoBA, HiP).

Static patterns: the foundational vocabulary

The field's vocabulary was set by four papers. **Sparse Transformer** (Child, Gray, Radford, Sutskever, OpenAI, 2019, arXiv:1904.10509) demonstrated that strided/fixed sparse attention preserves quality at sub-quadratic cost. **Longformer** (Beltagy, Peters, Cohan, AllenAI, 2020) introduced the canonical local-window-plus-global-tokens pattern with a CUDA kernel for sliding window. **BigBird** (Zaheer et al., NeurIPS 2020) added random connections to that backbone with a theoretical universality argument. **LongNet** (Ding, Ma, Dong et al., MSR, 2023) introduced exponentially-growing dilated attention scaling to a billion tokens, though its training-only nature means it is rarely deployed at inference unless the model was pretrained with it. (arXiv + 4)

The single most important empirical discovery for inference-time prefill came from **StreamingLLM** (Xiao, Tian, Chen, Han, Lewis, MIT/Meta, ICLR 2024). **The paper showed that the first few tokens of a sequence absorb disproportionate attention probability — "attention sinks" — and that keeping them in a sliding-window cache stabilizes streaming inference up to 4 M tokens with a 22.2× speedup over recompute baselines. Every subsequent dynamic method that talks about the "A-shape" pattern (sink + recent window) is descended from this observation.** **Star Attention** (NVIDIA, Acharya, Jia, Ginsburg, 2024) extends the idea to distributed inference: a two-phase scheme in which each shard processes its block with an "anchor" sink, then queries aggregate globally, achieving up to 11× speedup with under 2% drop on **RULER**. (GitHub + 5)

Dynamic prefill methods: pattern detection and query-aware selection

The single most influential prefill paper of 2024 is **MInference 1.0** (Jiang, Li, Zhang, Wu, Luo, Ahn, Han et al., Microsoft, NeurIPS 2024 Spotlight, arXiv:2407.02490). It identifies three canonical attention-head patterns — **A-shape, Vertical-Slash, and Block-Sparse** — and assigns each head one pattern offline via calibration; at inference, the sparse indices for that

pattern are computed per-prompt with custom Triton kernels. The headline result: **10× prefill speedup at 1 M tokens on A100** for LLaMA-3-1M, GLM-4-1M, Yi-200K, Phi-3-128K, and Qwen2-128K, with under 2% drop on **RULER, InfiniteBench**, NIAH, and PG-19. MInference defined the pattern vocabulary used by every 2025 follow-up and is the canonical baseline against which newer methods compare. (Microsoft + 2 + 8)

FlexPrefill (Lai, Lu, Luo, Ma, Zhou, ByteDance + PKU + HKU, ICLR 2025 Oral, arXiv:2502.20766) replaces MInference's offline head→pattern assignment with a per-prompt classifier: Jensen-Shannon divergence between sampled attention rows decides whether each head is "Diverse" (needs query-specific estimation) or "Structured" (uses a predefined pattern), and a cumulative-attention threshold $\tau \approx 0.9$ selects the minimal index set. The result is **Pareto-better than MInference on RULER** at matched sparsity. **XAttention** (Xu, Xiao, Huang, Guo, Han, MIT HAN Lab, ICML 2025, arXiv:2503.16428) reaches a similar accuracy frontier with a much cheaper proxy: **the sum of antidiagonal values in a 64×64 block is a powerful estimator of block importance because antidiagonals intersect both vertical and slash patterns**. XAttention is fully training-free and reports up to **13.5× attention speedup** with comparable accuracy on RULER, LongBench, and video benchmarks (VideoMME, VBench). (ICLR + 5)

SpargeAttention (Zhang, Xiang, Huang, Wei, Xi, Zhu, Chen, THU, ICML 2025) is notable for generality: it is universal and training-free, layering an attention-map predictor and an online softmax-aware filter on top of SageAttention's quantized kernel, achieving 4–7× speedup not only on language but on **Flux, SD3.5, Mochi, and CogVideoX** for image and video generation. **HiP / InfiniteHiP** (Lee et al., 2024–2025) takes a hierarchical-pruning approach with $O(T \log T)$ time and $O(T)$ memory, scaling pretrained LLMs to **3 M tokens on a single GPU** when combined with KV-cache offloading. (Hugging Face + 4)

A second branch fine-tunes a lightweight gate while freezing the base model. **SeerAttention** (Gao, Zeng, Du, Cao et al., HKU + UW + MSR, 2024–2025) attaches an MoE-style learnable gate to each attention layer that pools Q and K, multiplies them through small linear layers to produce **block-level gating scores**, and selects via top-k or threshold. The gate is trained by **self-distillation** to match the 2D max-pool of the model's own dense attention map. With YaRN at 32K, SeerAttention achieves 90% sparsity at **5.67× speedup over FlashAttention-2** with minimal perplexity loss; SeerAttention-R extends to decoding with a TileLang kernel reaching **9× speedup over FlashAttention-3 on H100**. DeepSeek's official DSA (V3.2) appears to inherit this self-distillation pipeline. (OpenReview + 5)

Several decoding-side methods cross-pollinate prefill design. **Quest** (Tang, Zhao, Xiao, Han et al., MIT HAN Lab, ICML 2024) introduced the **query-aware page-criticality** principle: store min/max key per KV-cache page, use the current query to estimate per-page upper-bound attention mass, load only top-K pages — a 7.03× attention and 2.23× E2E latency reduction. **Twilight** (Lin, Tang et al., NeurIPS 2025 Spotlight) is a composable wrapper: it adds a hierarchical top-p (nucleus) pruner on top of any base sparse algorithm, reporting up to 98% redundant-token pruning. **TriForce** (Sun, Chen, Yang, Tian, Chen, CMU + Meta, COLM 2024) uses sparse retrieval-based KV (~3% of full cache) as a draft inside hierarchical speculative decoding for 2.31× speedup on Llama2-7B-128K and 7.72× with offloading on dual RTX

arxiv + 6

arXiv + 3

arXiv arxiv

Method	Training	Selection	Granularity	Speedup	Notes
StreamingLLM	Free	Q-agnostic	Token (sink + window)	22.2× vs recompute	Defined "attention sinks"
MInference 1.0	Free	Q-aware (within head)	Per-head pattern	10× prefill @ 1M	Canonical post-hoc baseline

FlexPrefill	Free	Q-aware	Head + block	Pareto >	JSD pattern detection
		per-prompt		MI	Inference
XAttention	Free	Q-aware	Block (64×64)	13.5× attention	Antidiagonal-sum proxy
SpargeAttn	Free	Q-aware	Block	4–7×	Universal: text/image/video
SeerAttention	Gate finetune	Q-aware	Block	5.67× FA-2 @ 90% sparsity	Self-distillation gate
Quest	Free	Q-aware	KV page	7.03× attn / 2.23× E2E	Decode-focused, query-aware origins
HiP / InfiniteHiP	Free	Q-aware	Hierarchical block	3M tok / GPU	Tree top-k
NSA	Native pretrain	Q-aware	Block (3 branches)	9× fwd / 11.6× decode	DeepSeek; ACL'25 Best Paper
MoBA	Native pretrain	Q-aware	Block (MoE router)	Production at Kimi	NeurIPS'25 Spotlight

KV cache compression: five mostly-orthogonal axes

KV-cache compression has crystallized into five axes that frequently compose: **token-level eviction**, quantization, low-rank/merging, cross-layer sharing, and hybrid retrieval. Most early work (H2O, SnapKV, KIVI) is formally a decode-time optimization; the prefill-relevant question is whether compression can happen *online* during prompt encoding without seeing the future query.

Token-level eviction and selection

H2O (Heavy-Hitter Oracle) (Zhang et al., NeurIPS 2023) is foundational: it observes that a small set of "heavy hitter" tokens carries most accumulated attention and formulates eviction as a dynamic submodular problem with theoretical guarantees, retaining 20% of KVs at near-full quality and reporting up to **29× throughput** vs. DeepSpeed/Accelerate. **StreamingLLM**'s sink + window pattern, discussed above, is the second pillar; together H2O and **StreamingLLM** define the "keep recent + keep some important early/heavy tokens" template that everything later refines. **Scissorhands** (Liu et al., NeurIPS 2023) adds a "persistence of importance" hypothesis with probabilistic retention, giving 5× memory reduction by itself and 20× combined with 4-bit quantization. (arXiv +2 + 2)

The prefill-relevant breakthrough is **SnapKV** (Li et al., NeurIPS 2024): a small "observation window" at the end of the prompt votes via mean/max-pooled attention scores for which KVs to keep per-head. SnapKV achieves **3.6× generation speedup**, **8.2× memory efficiency** at

16K input, and 380K context on a single A100, becoming the de-facto prefill compression baseline. Crucially, SnapKV's vote implicitly assumes the user query is in the observation window, which makes it strong on RULER S-NIAH but weaker on **multi-key NIAH** and harder still in shared-prefix multi-turn settings — the key limitation flagged by RazorAttention and SCBench (Microsoft, ICLR 2025). (arXiv + 3)

PyramidKV (Cai et al., 2024) and **PyramidInfer** (Yang et al., ACL Findings 2024) add **layer-wise budget allocation**: lower layers retain more KVs (information is dispersed) and upper layers fewer (information is focused). **PyramidKV matches full KV at 12% retention on LongBench; PyramidInfer compresses during prefill (not after), giving 2.2× throughput and 54% less GPU memory.** **AdaKV / Ada-SnapKV** (Feng, Lv et al., NeurIPS 2025) prove a theoretical loss upper bound and add head-wise adaptive budgeting on top. **FastGen** (Ge, Zhang et al., ICLR 2024) profiles each head and assigns it to local-only / sink-only / column-sparse / dense policies. **Keyformer** (Adnan et al., MLSys 2024) uses a Gumbel-noise-regularized score function; **HeadKV** (Fu et al., ICLR 2025) and **DuoAttention** (Xiao et al., 2024) push head-level allocation further by explicitly identifying retrieval vs. streaming heads. (arXiv + 10)

Quantization

KIVI (Liu, Yuan, Jin et al., ICML 2024) is the structural template: **per-channel quantization for keys** (K outliers concentrate on channels) and **per-token for values** (streaming-friendly), giving tuning-free 2-bit KV with 2.6× peak memory reduction, up to 4× larger batch, and 2.35–3.47× throughput. **KVQuant** (Hooper, Kim et al., NeurIPS 2024) refines this with pre-RoPE per-channel K quantization, sensitivity-weighted non-uniform datatypes, and dense+sparse outlier isolation, reaching **3-bit with under 0.1 perplexity degradation** and supporting 1M-context LLaMA-7B on a single A100. **GEAR** (Kang et al., 2024) decomposes KV into low-bit quantized + low-rank residual + sparse outliers — a more sophisticated approximation that retains 97% of FP16 GSM8K accuracy at 4×. **Atom** (Zhao, Lin et al., MLSys 2024) is the first fully-low-bit serving recipe (W4A4KV4) with **7.7× throughput vs. FP16.**

(arXiv + 7)

Subsequent variants (**ZipCache, MiKV, IntactKV, SKVQ, QAO, Coupled Quantization**) each contribute one structural insight: ZipCache's normalized attention score corrects for lower-triangle masking; MiKV demotes unimportant tokens to low bits rather than evicting them; SKVQ keeps a recent FP16 window for accuracy with 2-bit K, 1.5-bit V supporting 1M context on 80 GB; CQ exploits inter-channel dependence to reach **1 bit per channel**. The cumulative practical lesson: **4-bit KV is essentially lossless on long-context benchmarks**, 3-bit is near-lossless with outlier tricks, and below 3 bits requires structural decomposition. (arxiv + 5)

Low-rank, merging, and cross-layer architectural compression

CaM (Cache Merging) (Zhang et al., ICML 2024), **KVMerger** (Wang et al., 2024), and **D2O** (Wan, Wu et al., ICLR 2025) replace eviction with merging: instead of dropping unimportant KVs, blend them into retained ones via attention-weighted or similarity-weighted

aggregation. **Eigen Attention** (Saxena et al., EMNLP Findings 2024) projects K, Q, V into a

low-rank eigenbasis for 40% size and 60% latency reduction. **LESS** (Dong et al., ICML 2024) keeps a constant-size low-rank state that recurrently absorbs the residual between full attention and a sparse policy. The 2025 wave (**Palu, Loki, MatryoshkaKV, ShadowKV, LoRC, EliteKV, OjaKV**) increasingly converges toward MLA-like low-rank K/V projections on pretrained MHA models. (ICML + 5)

Cross-layer sharing is conceptually simpler. **MQA** (Shazeer 2019) and **GQA** (Ainslie et al. 2023) shrink the head dimension and are now baseline architecture in Llama, Mistral, Qwen, and Phi. **CLA (Cross-Layer Attention)** (Brandon, Mishra, Nrusimha et al., NeurIPS 2024) extends this depth-wise: groups of layers all attend to one shared KV set, giving 2× smaller KV than MQA at parity. **MiniCache** (Liu et al., NeurIPS 2024) does the same training-free by interpolating directions of adjacent-layer KVs while keeping magnitudes, achieving 5.02× compression combined with 4-bit. **YOCO (You Only Cache Once)** (Sun, Dong, Zhu et al., NeurIPS 2024) reorganizes the model into a self-decoder that produces a single global KV reused by a cross-decoder, giving **L-fold KV reduction** (e.g., 32× on a 32-layer model), 9.38× less memory than GQA at 1M, and roughly 10× faster prefill via early exit. (arXiv + 6)

MLA: the architectural shift that reframes the field

Multi-Head Latent Attention (MLA) in **DeepSeek-V2** (May 2024, arXiv:2405.04434) and unchanged in **DeepSeek-V3** (Dec 2024, arXiv:2412.19437) is the most consequential architectural change of the period. MLA jointly low-rank-compresses K and V into a single latent vector c_{KV} (dimension $d_c \approx 4 \times d_{head}$) that is what gets cached; per-head K and V are reconstructed at attention time by up-projection matrices that can be absorbed into Q and output projections so no extra matmul is needed at inference. The position-encoding subtlety is the **decoupled RoPE branch**: because RoPE doesn't commute with arbitrary low-rank projection, MLA splits each head into a content portion that goes through low-rank compression *without* RoPE and a small RoPE key per token shared across heads, combined at attention time. (Vitalab + 3)

The numbers are dramatic. MLA shrinks KV cache by **~93% (~14×) vs. MHA**, gives **5.76× max generation throughput**, saves 42.5% in training cost, and — uniquely among compression methods — **exceeds MHA quality** at the same parameter count. **MHA→MLA conversion** (TransMLA, Feb 2025; MHA2MLA, arXiv:2502.14837) shows existing GQA models can be retrofitted with 0.3–0.6% additional training data, recovering 92% KV **reduction with only 0.5% LongBench drop on Llama-2-7B**. MLA is now the architecture for **DeepSeek-V2/V3/R1, V3.2, Kimi K2/K2.5/K2.6**, and increasingly considered the GQA replacement at frontier scale. The downstream consequence: **post-hoc KV compression motivations weaken sharply** when the architecture has already done the work, and remaining headroom is mostly in (a) MHA/GQA legacy checkpoints, (b) further quantization on top of MLA's latent, and (c) prefill-compute reduction (which MLA does not address).

(arXiv) (arXiv)

Approach

KV reduction

Quality vs MHA

Training?

Approach	KV reduction	Quality vs MHA	Training
MHA	0%	baseline	—
MQA / GQA	4-8×	small drop	Pretrained-in
MLA	~14× (~93%)	better than MHA	Pretrained-in
H2O / StreamingLLM (eviction)	5×, irreversible	NIAH degradation	Training-free
KIVI / KVQuant 4-bit	4×	≈ lossless	Training-free
RazorAttention	3.3×	near-lossless	Training-free
ThinK + KIVI	2.8× peak	near-identical	Training-free
MiniCache + 4-bit	5×	≈ lossless	Training-free

Hybrid systems: where sparse attention meets KV compression

Several systems combine both directions. **RazorAttention** (Tang, Lin et al., ICLR 2025, arXiv:2407.15891) identifies a small set of "**retrieval heads**" that attend across the full context and keeps full KV only for them; non-retrieval heads keep recent + sinks plus a learned **compensation token** that absorbs information from dropped KVs. The result is **>70% KV reduction with no noticeable performance loss** and FlashAttention compatibility, validated to 1024K NIAH. **DuoAttention** is a contemporary variant with optimization-based head labeling. **ThinK** (Xu et al., NeurIPS 2024) is orthogonal: it prunes the **channel dimension** of K cache via a query-driven importance criterion, contributing **>20% additional memory savings on top of any eviction baseline** (ThinK + KIVI achieves 2.8× peak memory reduction). (arXiv + 6)

InfLLM (Xiao, Han et al., NeurIPS 2024) treats distant context as block memory with representative tokens; queries retrieve top-k blocks per step, letting Mistral-7B-Inst and Llama-3-8B-Inst handle up to 1024K tokens without fine-tuning. **RetrievalAttention** (Microsoft, 2024) goes further and treats the KV cache as a **vector database** offloaded to CPU with attention-aware ANNS that handles the OOD distribution gap between Q and K, reaching near-full-attention quality while accessing **1-3% of vectors** and supporting 128K context on a single RTX 4090. **RetroInfer** (2025) scales this with a wave-index and GPU/CPU co-execution. **FastKV** (Feb 2025) explicitly decouples context reduction from KV compression with a "Token-Selective Propagation" layer, hitting 1.82× prefill and 2.87× decode speedup. **RocketKV** combines RazorAttention-style retrieval-head retention with token-level eviction for two-stage compression. (arXiv +2 + 5)

DeepSeek Sparse Attention (DSA) in **DeepSeek-V3.2** (Sept 2025; arXiv:2512.02556 for the full release) is the production culmination of this direction. DSA is **instantiated under MLA** (so the architecture is already KV-compressed) and adds a **lightning indexer** that computes cheap index scores plus **fine-grained token-level top-k selection** — finer than NSA's block-

level selection. This enables the first fine-grained sparse attention deployed at frontier scale with **~50% lower API cost** for long-context calls, with Day-0 vLLM and SGLang support on H100/H200/B200. [arXiv](#)

Benchmarks and methodology critiques

The benchmarks used to evaluate these methods have themselves become a research subject. **Needle-in-a-Haystack (NIAH)** (Greg Kamradt, 2023) — insert a distinctive sentence at varying depths in a long Paul Graham haystack and ask the model to retrieve it — became the default after Anthropic, Google, and OpenAI used it in launch announcements, but the consensus by 2025 is that **it is saturated and misleading**. NIAH measures verbatim retrieval, not reasoning over long context, and the lexical distinctiveness of the needle ("magic number") relative to the haystack lets the model solve it via surface pattern matching. **NoLiMa** (Modarressi et al., ICML 2025) eliminates literal overlap and demonstrates GPT-4o falling from 99.3% baseline to 69.7% at 32K, with 11 of 13 models below 50% of base score. **BABILong** (Kuratov, Bulatov et al., NeurIPS 2024) shows GPT-4 effectively uses only ~10% of its 128K window on multi-fact reasoning. The takeaway: **effective context is far shorter than advertised**, and methods that pass NIAH may quietly fail real workloads. [Nrehiew](#) [arxiv](#)

RULER (Hsieh, Sun, Krizan, Acharya, Rekesh, Jia, Zhang, Ginsburg, NVIDIA, COLM 2024, [arXiv:2404.06654](#)) is now the standard. It expands NIAH into 13 synthetic tasks across four categories: retrieval (single, multi-key, multi-value, multi-query NIAH), multi-hop variable tracking, common/frequent words extraction, and long-context QA over SQuAD/HotpotQA. The original paper's most cited finding: **only 4 of 10 models that claim 32K-token windows actually handle 32K**, and RULER scores typically run 10–25 points below NIAH single-needle at the same length. [arXiv + 4](#)

LongBench (Bai et al., ACL 2024) added 21 bilingual realistic tasks across QA, summarization, few-shot ICL, and code; **LongBench-v2** (Dec 2024) extended to 503 multiple-choice questions over 8K–2M-word contexts with expert baseline ~53.7% and o1-preview at only 57.7%, refocusing evaluation on **deep reasoning over retrieval**.

InfiniteBench (Zhang et al., ACL 2024, OpenBMB) was the first benchmark with average length >100K, spanning summarization, QA, code execution, math, and retrieval tasks.

[GitHub](#) [Longbench2](#)

The methodological capstone is **HELMET (How to Evaluate Long-context Models Effectively and Thoroughly)** (Yen, Gao, Hou et al., Princeton + Intel, ICLR 2025, [arXiv:2410.02694](#)). HELMET makes six concrete critiques of prior benchmarks: limited application coverage, insufficient context lengths, unreliable F1/ROUGE metrics, base-model incompatibility, NIAH saturation, and **poor inter-category correlation** — synthetic recall is a weak predictor of summarization, ICL, and generation-with-citations performance. HELMET evaluates 51+ frontier LCLMs across seven task categories (synthetic recall, RAG, passage re-ranking, long-document QA, summarization, many-shot ICL, generation with citations) up to 128K with model-based metrics and finds **RAG is the single best correlate** with downstream performance — a recommendation now adopted by Stanford's HELM Long Context

leaderboard. Workshop benchmarks like **Loong**, **Marathon**, **NoLiMa**, **OneRuler**, **LV-Eval**, **LongProc** fill specific gaps (multi-doc necessity, multilingual, long-output procedural).

Production deployment: what actually ships

Research-to-production transition is sharply selective. The universal stack across **vLLM**, **SGLang**, **TensorRT-LLM**, and **lmdeploy** as of mid-2026 is: **PagedAttention + GQA/MLA + FP8 (or INT8) KV cache + chunked prefill + automatic prefix caching + dynamic sparse prefill (MInference-style) + natively-sparse decode (DSA-style)**.

vLLM (Kwon et al., SOSP 2023) added MLA support in v0.7.1 (Feb 2025) with matrix-absorption optimizations and CUTLASS Block FP8, yielding $\sim 3\times$ throughput, $10\times$ KV-cache token capacity, and scaling $54.5K \rightarrow 512K$ tokens on commodity interconnects. **MInference** sparse kernels (vertical-slash and block-sparse) merged in April 2025; **DeepSeek V3.2 DSA** had Day-0 support in September 2025. **SGLang** (LMSYS, 2023) is built on **RadixAttention** for automatic prefix sharing, achieves up to $5\times$ throughput on shared-prefix workloads, has a **HiCache** three-tier (GPU/host/distributed) hierarchy yielding $\sim 6\times$ throughput and 80% TTFT reduction, and offers an `(nsa)` attention backend with FlashMLA sparse + FlashAttention-3 Sparse for V3.2 (default $k=2048$ selected tokens). **TensorRT-LLM** ships paged KV, FP8/INT8 KV ($1.09\text{--}1.45\times$ throughput per SqueezeBits), MLA, MTP, and NVFP4 + Sparse Attention paths for V3.2 on Blackwell. **lmdeploy** offers per-head/per-token asymmetric INT8/INT4 KV (`(quant_policy=8)/(4)`) and the ICLR 2026 **TurboQuant** scheme (`(quant_policy=42)`, $K=4\text{-bit QJL} + V=2\text{-bit MSE}$) for $\sim 5\times$ KV reduction at 7-8% throughput overhead. `(Red Hat + 5)`

What did **not** make it to production tells the same story. **H2O**, **SnapKV**, **Scissorhands**, **PyramidKV**, **FastGen**, **KIVI** remain research-only in mainline vLLM/SGLang/TRT-LLM despite hundreds of citations. The vLLM RFC #5751 cites three reasons: long-decoding regressions on reasoning models, insufficient generality across workloads, and **reluctance to permanently discard tokens that may be needed for future queries in shared-prefix multi-turn serving**. KIVI's 2-bit scheme has been **superseded by FP8 + TurboQuant in production**. The lesson is consistent: **architectural sparse attention and quantization win in production, while attention-score-based eviction methods stay in research**.

Open problems

Several problems remain unsolved by mid-2026. **Unified prefill+decode sparsity without retraining** — most training-free methods are either prefill-only (MInference, FlexPrefill, XAttention) or decode-only (Quest, Twilight). NSA and MoBA unify both but require pretraining; **whether a training-free method can be lossless in both phases is open**. **Query-agnostic prefill compression for retrieval workloads** is the central conceptual gap:

SnapKV's observation window covertly assumes the query is known at compression time, an assumption that fails in shared-prefix multi-turn dialog and RAG with reusable prefix caches. RazorAttention and DuoAttention sidestep it via retrieval-head identification; RetrievalAttention and InfLLM sidestep it via offloading; **none yet match full-KV on the**

hardest multi-key NIAH at extreme compression.

Reasoning-time decoded-token compression is a new frontier exposed by R1-distilled and o-series models that generate thousands of CoT tokens. Recent work ("Hold Onto That Thought," 2025) shows prefill-tuned compressors break down on long reasoning. **Hardware portability** of sparse kernels is also limited: Hopper-specific FA-3 / NSA / SeerAttention-R kernels do not yet target Blackwell or AMD MI300, and compression performance is not portable. **Theoretical understanding** is still nascent — the Optimizing-MoBA SNR analysis is one of the first principled accounts of sparse-attention block routing, but a general theory connecting sparsity ratio, block size, head dimension, and accuracy retention is missing. **Multi-modal sparsity** for video (MMInference's Grid pattern, VideoNSA, Mixture of Contexts at ICLR 2026) is in its infancy: video diffusion (HunyuanVideo, CogVideoX) demands new pattern families, and quality on long video remains fragile. Finally, **benchmark methodology** itself remains contested — even RULER and HELMET are early in the lifecycle of "benchmarks that distinguish frontier long-context behavior from advertised context windows."

Conclusion: from inference patches to architectural primitives

The five years from 2020 to 2025 saw long-context efficiency move through three regimes. The first (2020–2022, Longformer/BigBird/LongNet) tried to **train sparsity in** but worked at small scale on specialized models. The second (2022–2024, StreamingLLM/H2O/MInference/KIVI/SnapKV) **accepted dense pretrained models as given and patched them at inference time** with eviction, quantization, and pattern detection — generating most of the cited literature but bumping into a quality wall (irreversible token loss, query-agnosticism, kernel-fragmented sparsity). The third (2024–2026, MLA/NSA/MoBA/DSA) **returned to architectural integration** but now at frontier scale with hardware-aligned kernel design, validated by deployment in DeepSeek-V2/V3/V3.2 and Kimi K2.

The most striking signal is what production actually deploys: not H2O or SnapKV, but **PagedAttention, GQA, MLA, FP8 KV, MInference, and DSA** — a stack dominated by architectural choices and dense-but-clever kernels rather than attention-score heuristics. The 2026 picture is convergent. Architectural compression (MLA) plus architectural sparsity (NSA/DSA/MoBA) plus quantization (FP8/INT8/TurboQuant) plus retrieval-augmented offloading (RetroInfer, HiCache) compose cleanly because each operates on a different axis. The remaining post-hoc inference research is not dead but increasingly serves as a *complement* to this architectural backbone, particularly for legacy MHA/GQA checkpoints and for prefill-compute reduction (orthogonal to KV memory). The open question is whether the next architectural primitive — perhaps a unified retrieval-aware sparse-attention layer that subsumes NSA, MoBA, and RetrievalAttention — will emerge from frontier labs (DeepSeek's V4 sparse-attention previews suggest yes) or from academic research catching back up.